A REPORT ON LAB EXPERIMENT

ON

Air Quality Index (AQI) Predictor

A Course Project report Submitted in partial fulfillment of the Academic requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

CSE(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

Submitted by

S.Shirisha	23D41A66G9
T.Harini	23D41A66H0
T.Rishwanth	23D41A66H1
U.Navya	23D41A66H2
U.Akash	23D41A66H3

Under the esteemed guidance of

Mrs.Mosangi Uppamma

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

SRI INDU COLLEGE OF ENGINEERING & TECHNOLOGY(Autonomous)

(Approved by AICTE, New Delhi & Permanently Affiliated to JNTU Hyderabad,Sheriguda (V), Ibrahimpatnam (M),R.R.Dist-501 510)

2025-2026

SRI INDU COLLEGE OF ENGINEERING & TECHNOLOGY

(An Autonomous Institute)
(Approved by AICTE, New Delhi & Permanently Affiliated to JNTU Hyderabad)

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

CERTIFICATE

This is to certify that the Project entitles **"Air Quality Index(AQI) Predictor"** is submitted in partial fulfilment for the award of **Degree of Bachelors of Technology in CSE(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)** of Sri Indu College of Engineering and Technology,Hyderabad for the academic year 2026-2027.This is a record of the project work done by **S.Shirisha(23D41A66G9),T.Harini(23D41A66H0),T.Rishwanth(23D41A66H1),U.Navya (23D41A66H2),U.Akash(23D41A66H3)** which has been approved as it satisfies academic requirements in respect in respect of the work prescribed for **IV YEAR,I-SEMESTER of B.Tech course.**

Mrs.MOSANGI UPPAMMA
(Internal Guide)

G.UMA MAHESWARI
(Head of the Department)

TABLE OF CONTENTS

SNO	TOPIC	PAGE NO
1	ABSTRACT	1
2	INTRODUCTION	2
3	EXISTING SYSTEM	3
4	PROPOSED SYSTEM	4
5	HARDWARE AND SOFTWARE REQUIREMENTS	5
6	IMPLEMENTATION	6
7	RESULT	12
8	CONCLUSION	13

ABSTRACT

The **Air Quality Index (AQI) Predictor** is a machine learning-based project developed to classify air quality levels using environmental data and provide users with a better understanding of air pollution and its potential effects on human health. Air pollution is one of the major environmental challenges, as it can lead to respiratory problems, cardiovascular diseases, and other health complications. Monitoring air quality and understanding pollution levels are essential for protecting public health and creating awareness about environmental conditions. This project focuses on predicting air quality categories using important environmental parameters, particularly PM2.5 concentration, hour of the day, and month of the year. PM2.5 stands for Particulate Matter 2.5 and refers to tiny airborne particles with a diameter of 2.5 micrometers or less. These particles can originate from vehicle emissions, industrial activities, smoke, and the burning of fuels. Due to their extremely small size, they can penetrate deep into the lungs and may pose serious health risks. Therefore, PM2.5 is an important parameter for understanding air pollution and estimating its possible impact on human health.

The project uses machine learning algorithms, including Decision Tree, K-Nearest Neighbors (KNN), and Gaussian Naive Bayes, to classify air quality into six categories: Good, Satisfactory, Moderate, Poor, Very Poor, and Severe. A historical dataset containing environmental records is used to train and evaluate the models. The system is developed using Python and Scikit-learn, while Streamlit is used to create an interactive web application through which users can enter environmental parameters and obtain predicted air quality categories. Plotly is used to present graphical visualizations and comparisons of model performance, helping users understand the results more easily. The application also provides health-related advisories and precautionary recommendations based on the predicted category. By combining machine learning, data visualization, and an accessible user interface, the Air Quality Index (AQI) Predictor aims to simplify air quality classification, improve public awareness of pollution levels, and demonstrate the practical application of data mining and machine learning techniques in environmental analysis.

INTRODUCTION

Air pollution is a major environmental concern that affects human health, the environment, and the overall quality of life. Increasing industrialization, urbanization, vehicle emissions, construction activities, and the burning of fossil fuels contribute to the release of harmful pollutants into the atmosphere. Continuous exposure to polluted air can cause respiratory illnesses, cardiovascular problems, and other health-related complications. Therefore, monitoring air quality and providing understandable information about pollution levels are important for reducing exposure to harmful pollutants and promoting public awareness. The Air Quality Index (AQI) is commonly used to represent air quality through categories such as Good, Satisfactory, Moderate, Poor, Very Poor, and Severe. One important pollutant associated with air pollution is PM2.5, which stands for Particulate Matter 2.5. It consists of extremely small airborne particles with a diameter of 2.5 micrometers or less. These particles may be produced by vehicle exhaust, industrial emissions, smoke, dust-related activities, and the burning of fuels. Because PM2.5 particles are very small, they can travel deep into the respiratory system and may cause adverse health effects. Understanding PM2.5 concentration is therefore important when assessing air pollution and predicting air quality conditions.

The **Air Quality Index (AQI) Predictor** is designed to apply machine learning techniques to classify air quality based on environmental data. The system uses PM2.5 concentration, hour of the day, and month of the year as input features to predict the corresponding air quality category. It implements three machine learning algorithms: Decision Tree, K-Nearest Neighbors (KNN), and Gaussian Naive Bayes. The models are trained and evaluated using a historical dataset containing air-quality records, allowing their classification performance to be compared. Python and Scikit-learn are used for data processing and model development, while Streamlit provides an interactive web interface for entering input values and viewing prediction results. Plotly supports graphical representation and comparison of model performance, and the application provides health advisories based on the predicted air quality category. The main objective of this project is to make air quality classification easier to understand and accessible through a simple web-based application. It also demonstrates how data mining and machine learning can be applied to environmental data to identify air quality categories, visualize results, and support public awareness. The system currently focuses on model-based prediction using user-provided inputs rather than direct, real-time monitoring through physical sensors.

EXISTING SYSTEM

The existing system for air quality monitoring focuses on measuring pollutant concentrations, collecting environmental data, and classifying air quality based on established Air Quality Index (AQI) standards. Air quality monitoring stations and environmental sensors measure pollutants present in the atmosphere, helping identify pollution levels and their possible effects on human health. The Air Quality Index categorizes air quality into six levels: Good, Satisfactory, Moderate, Poor, Very Poor, and Severe. These categories help people understand environmental conditions and take necessary precautions to reduce exposure to air pollution.

Limitations of the Existing System:

- Raw pollutant measurements may be difficult for ordinary users to interpret.
- Manual analysis and classification of environmental data can be time-consuming.
- Some systems provide limited interactive visualizations and model comparisons.
- Understanding the health implications of different pollution levels may require additional information.

To address these limitations, the proposed **Air Quality Index (AQI) Predictor** uses machine learning algorithms and an interactive web application to classify air quality, visualize results, compare model performance, and provide health-related recommendations. However, the current project uses user-provided environmental values and historical data rather than directly collecting real-time measurements from physical sensors.

PROPOSED SYSTEM

The proposed **Air Quality Index (AQI) Predictor** uses machine learning techniques to classify air quality based on environmental data. The system uses PM2.5 concentration, hour of the day, and month of the year to predict air quality categories. PM2.5 (Particulate Matter 2.5) refers to tiny airborne particles with a diameter of 2.5 micrometers or less, which can penetrate deep into the lungs and affect human health.

The proposed system uses historical air-quality data to train and evaluate machine learning models. It provides an interactive web application developed using Python, Scikit-learn, and Streamlit, allowing users to enter environmental parameters and view prediction results.

The major components of the proposed system are:

- **Data Preprocessing:** Prepares historical environmental data for machine learning.
- **Machine Learning Models:** Implements Decision Tree, K-Nearest Neighbors (KNN), and Gaussian Naive Bayes algorithms.
- **AQI Prediction:** Classifies air quality into Good, Satisfactory, Moderate, Poor, Very Poor, and Severe.
- **Interactive Interface:** Allows users to enter environmental values and view predicted results.
- **Data Visualization:** Uses Plotly to display graphs and compare model performance.
- **Health Recommendations:** Provides health advisories based on the predicted air quality category.

The proposed system aims to simplify air quality classification, improve the presentation of environmental data, and increase awareness of air pollution. The current implementation uses historical data and user-provided inputs rather than direct, real-time monitoring through physical sensors.

HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: 4 GB or above
- Hard Disk: 10 GB free space
- Operating System: 64-bit computer

Software Requirements

- Programming Language: Python
- Libraries: Pandas, NumPy, Scikit-learn
- Web Framework: Streamlit
- Visualization: Plotly
- IDE: Visual Studio Code
- Web Browser: Any Browser to view streamlit apps

IMPLEMENTATION

app.py:

```
import streamlit as st
import pandas as pd
import numpy as np
import base64
from pathlib import Path
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_recall_fscore_support, classification_report
import plotly.graph_objects as go
import joblib

st.set_page_config(
    page_title="Air Quality Index (AQI) Classifier",
    page_icon="🍃",
    layout="wide",
    initial_sidebar_state="expanded"
)

if "theme_mode" not in st.session_state:
    st.session_state["theme_mode"] = "Light"

with st.sidebar:
    st.markdown("### 🎨 **Theme**")
    theme_choice = st.radio(
        "Theme",
                        ["☀️", "🌙"],
        index=0 if st.session_state["theme_mode"] == "Light" else 1,
        horizontal=True,
        label_visibility="collapsed"
    )
```

```
        st.session_state["theme_mode"] = "Light" if theme_choice == "☀️" else "Dark"

is_dark = st.session_state["theme_mode"] == "Dark"

@st.cache_data
def get_base64_bg(relative_path: str) -> str:
    img_file = Path(__file__).resolve().parent / relative_path
    if img_file.exists():
        with open(img_file, "rb") as f:
            return base64.b64encode(f.read()).decode()
    return ""

light_bg_b64 = get_base64_bg("assets/air_light_bg.jpg")
dark_bg_b64 = get_base64_bg("assets/air_dark_bg.jpg")
active_bg = dark_bg_b64 if is_dark else light_bg_b64

if is_dark:
    text_main = "#f0f9ff"
    text_muted = "#94a9be"
    accent = "#38bdf8"
    accent_glow = "rgba(56, 189, 248, 0.4)"
    bg_sidebar = "rgba(9, 17, 28, 0.92)"
    bg_card = "rgba(13, 23, 38, 0.82)"
    border_color = "rgba(56, 189, 248, 0.25)"
    btn_bg = "rgba(18, 30, 48, 0.82)"
    btn_hover_bg = "rgba(28, 46, 74, 0.92)"
    input_bg = "rgba(13, 23, 38, 0.88)"
    card_shadow = "0 8px 32px rgba(0, 0, 0, 0.45)"
    hero_bg = "linear-gradient(135deg, rgba(15, 26, 44, 0.88) 0%, rgba(10, 36, 52, 0.82) 100%)"
    hero_shadow = "0 12px 36px rgba(0, 0, 0, 0.60)"
    plot_template = "plotly_dark"
else:
    text_main = "#0c2438"
    text_muted = "#335c75"
    accent = "#0284c7"
```

```
    accent_glow = "rgba(2, 132, 199, 0.25)"

    bg_sidebar = "rgba(255, 255, 255, 0.85)"
    bg_card = "rgba(255, 255, 255, 0.82)"
    border_color = "rgba(2, 132, 199, 0.18)"
    btn_bg = "rgba(255, 255, 255, 0.75)"
    btn_hover_bg = "rgba(224, 242, 254, 0.95)"
    input_bg = "rgba(255, 255, 255, 0.90)"
    card_shadow = "0 8px 32px rgba(2, 132, 199, 0.08)"
    hero_bg = "linear-gradient(135deg, rgba(240, 249, 255, 0.92) 0%, rgba(224, 242, 254, 0.86) 100%)"
    hero_shadow = "0 12px 36px rgba(2, 132, 199, 0.12)"
    plot_template = "plotly_white"

def get_ground_truth_category(pm: float) -> str:
    if pm <= 30:
        return "Good"
    elif pm <= 60:
        return "Satisfactory"
    elif pm <= 90:
        return "Moderate"
    elif pm <= 120:
        return "Poor"
    elif pm <= 250:
        return "Very Poor"
    else:
        return "Severe"

@st.cache_resource
def load_or_train_classification_models():
    model_path = Path(__file__).resolve().parent / "model.joblib"
        if model_path.exists():
        try:
            saved_data = joblib.load(model_path)
            if isinstance(saved_data, dict) and "models" in saved_data:
                return saved_data["models"], saved_data["summary_df"], saved_data["reports"]
```

```
    except Exception:
        pass

csv_path = Path(__file__).resolve().parent / "data" / "air-quality-india.csv"
if not csv_path.exists():
    csv_path = Path(__file__).resolve().parent / "dataset" / "air-quality-india.csv"

df = pd.read_csv(csv_path)
df["Category"] = df["PM2.5"].apply(get_ground_truth_category)

feature_cols = ["PM2.5", "Hour", "Month"]
X = df[feature_cols]
y = df["Category"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

dt = DecisionTreeClassifier(max_depth=8, random_state=42)
dt.fit(X_train, y_train)

nb = GaussianNB()
nb.fit(X_train, y_train)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

trained_models = {
    "Decision Tree": dt,
    "Naive Bayes": nb,
    "KNN": knn
}

metrics_summary = []
reports = {}
```

```
    for name, clf in trained_models.items():

        preds = clf.predict(X_test)
        acc = accuracy_score(y_test, preds)
        p, r, f1, _ = precision_recall_fscore_support(
            y_test, preds, average="weighted", zero_division=0
        )
        report_dict = classification_report(
            y_test, preds, output_dict=True, zero_division=0
        )

        metrics_summary.append({
            "Algorithm": name,
            "Accuracy": acc,
            "Precision": p,
            "Recall": r,
            "F1-Score": f1
        })
        reports[name] = report_dict

    summary_df = pd.DataFrame(metrics_summary)

    try:
        joblib.dump({
            "models": trained_models,
            "summary_df": summary_df,
            "reports": reports
        }, model_path)
    except Exception:
        pass

    return trained_models, summary_df, reports

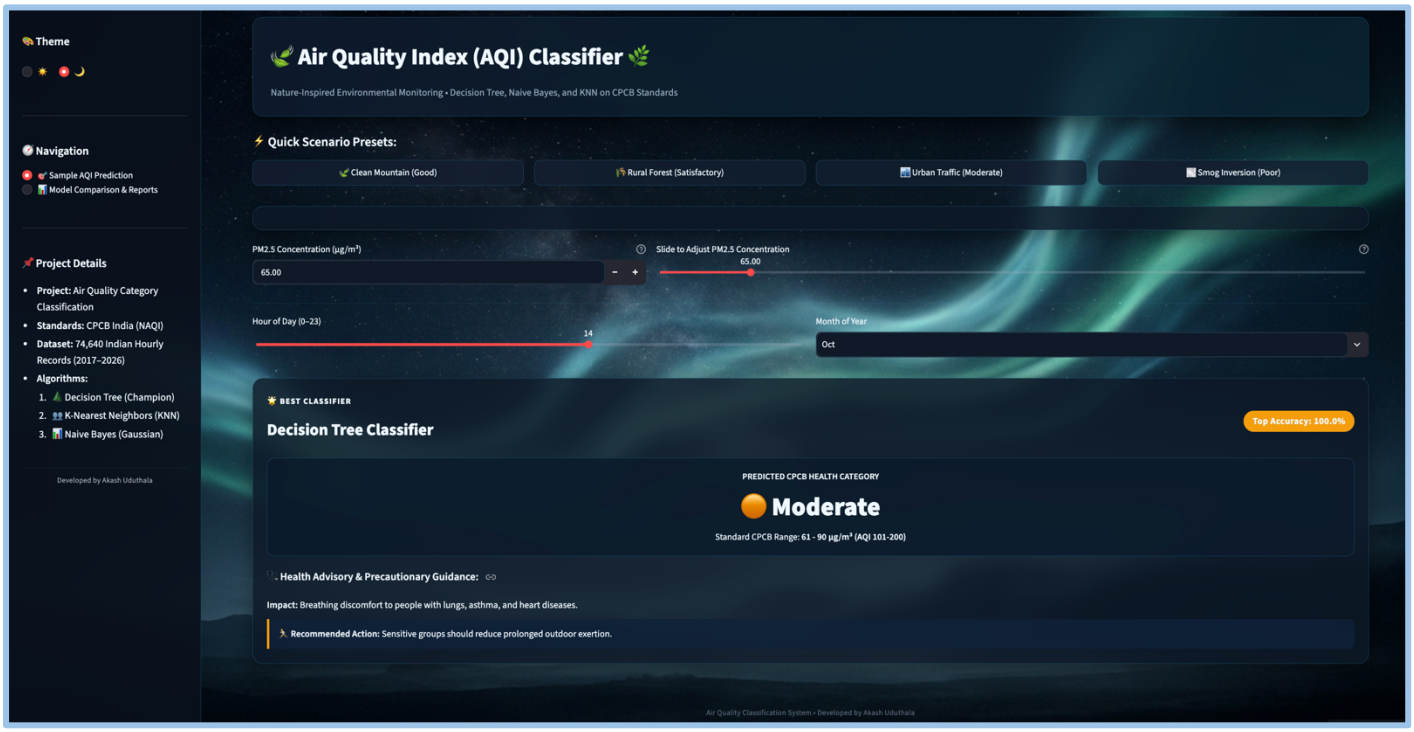
models_dict, summary_metrics_df, detailed_reports = load_or_train_classification_models()
```

```
with st.sidebar:
    st.divider()

    st.markdown("### 🧭 **Navigation**")
    selected_view = st.radio(
        "Navigation",
        [
            "🎯 Sample AQI Prediction",
            "📊 Model Comparison & Reports"
        ],
        index=0,
        label_visibility="collapsed"

    )
```

RESULT


Theme
Navigation
Sample AQI Prediction
Model Comparison & Reports
Project Details
Project: Air Quality Category Classification
Standards: CPCB India (NAQI)
Dataset: 74,640 Indian Hourly Records (2017–2026)
Algorithms:
1. Decision Tree (Champion)
2. K-Nearest Neighbors (KNN)
3. Naive Bayes (Gaussian)
Developed by Akash Uduthala
Air Quality Index (AQI) Classifier
Nature-Inspired Environmental Monitoring • Decision Tree, Naive Bayes, and KNN on CPCB Standards
Quick Scenario Presets:
Clean Mountain (Good)
Rural Forest (Satisfactory)
Urban Traffic (Moderate)
Smog Inversion (Poor)
PM2.5 Concentration (µg/m³)
65.00
Slide to Adjust PM2.5 Concentration
65.00
Hour of Day (0–23)
14
Month of Year
Oct
BEST CLASSIFIER
Decision Tree Classifier
Top Accuracy: 100.0%
PREDICTED CPCB HEALTH CATEGORY
Moderate
Standard CPCB Range: 61 - 90 µg/m³ (AQI 101-200)
Health Advisory & Precautionary Guidance:
Impact: Breathing discomfort to people with lungs, asthma, and heart diseases.
Recommended Action: Sensitive groups should reduce prolonged outdoor exertion.
Air Quality Classification System • Developed by Akash Uduthala


Theme
Navigation
Sample AQI Prediction
Model Comparison & Reports
Project Details
Project: Air Quality Category Classification
Standards: CPCB India (NAQI)
Dataset: 74,640 Indian Hourly Records (2017–2026)
Algorithms:
1. Decision Tree (Champion)
2. K-Nearest Neighbors (KNN)
3. Naive Bayes (Gaussian)
Developed by Akash Uduthala
Algorithms Performance Comparison
Evaluated on 14,928 held-out testing records (20% of 74,640 records) from Indian CPCB monitoring network (2017–2026).
Algorithm | Accuracy | Precision | Recall | F1-Score
Decision Tree | 99.99% | 99.99% | 99.99% | 99.99%
Naive Bayes | 96.06% | 96.10% | 96.06% | 96.06%
KNN | 99.49% | 99.49% | 99.49% | 99.49%
Model Comparison Chart
Accuracy (%)
Precision (%)
Recall (%)
F1-Score (%)
Score (%)
105
100
95
90
85
80
Decision Tree
Naive Bayes
KNN
Algorithm
Precision & Recall Reports by Class
Select Algorithm to inspect detailed classification report:
Decision Tree
AQI Category | Precision | Recall | F1-Score | Support (Samples)
Good | 100.00% | 100.00% | 100.00% | 4553
Satisfactory | 100.00% | 100.00% | 100.00% | 5958
Moderate | 100.00% | 100.00% | 100.00% | 2729
Poor | 100.00% | 100.00% | 100.00% | 754
Very Poor | 100.00% | 99.89% | 99.95% | 926

CONCLUSION

The Air Quality Index (AQI) Predictor successfully demonstrates the application of machine learning techniques for air quality classification using environmental data. The system uses important input parameters, including PM2.5 concentration, hour of the day, and month of the year, to predict air quality categories such as Good, Satisfactory, Moderate, Poor, Very Poor, and Severe. By converting pollution measurements into understandable categories, the system helps users interpret air quality conditions more easily and understand their potential effects on human health. The project implements three machine learning algorithms: Decision Tree, Naive Bayes, and K-Nearest Neighbors (KNN), allowing their performance to be evaluated and compared using accuracy and other evaluation metrics.

The application is developed using Python, Pandas, Scikit-learn, and Streamlit, providing a user-friendly and interactive interface for entering environmental data and viewing prediction results. It also uses data visualization to present model performance in a clear and understandable format. The system reduces the effort required to analyze raw pollution measurements and supports awareness of air pollution and its possible health risks. Although the current implementation relies on historical data and user-provided inputs rather than direct real-time sensor measurements, it provides a useful foundation for environmental data analysis. In the future, the project can be enhanced by integrating real-time air quality sensors, including additional pollutants such as PM10, nitrogen dioxide, and carbon monoxide, and implementing forecasting models to estimate future pollution levels. These improvements could make the system more comprehensive and useful for environmental monitoring, public awareness, and informed decision-making.